UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica Industrial y Automática

PROYECTO DE INFORMÁTICA INDUSTRIAL

DESARROLLO DEL VIDEOJUEGO ARCHON MEDIANTE PROGRAMACIÓN ORIENTADA A OBJETOS EN C++

RANCHO

Julián Osiniri Juárez

Álvaro Pérez Ruano

Miguel Vázquez Aguado

Jaime Ladero Blanco

David San Román González

Madrid, Mayo de 2026

ÍNDICE

1. INTRODUCCIÓN 2

1.1. Contexto 2

1.2. Origen de "Rancho" 2

1.3. Objetivos del proyecto 3

2. MANUAL DE INSTRUCCIONES 4

2.1. Requisitos del sistema y dependencias 4

2.2. Guía de inicio 4

2.3. Controles 4

2.4. Reglas del juego 5

3. ARQUITECTURA DEL SOFTWARE (POO) 5

3.1. Desacople entre lógica y motor gráfico 5

3.2. Patrones de diseño 5

3.3. Estructura de clases 5

4. LÓGICA DE JUEGO 5

4.1. Fase estratégica (Tablero) 5

4.2. Fase de acción (Arena) 5

4.3. Las piezas (Animales) 6

5. METODOLOGÍA Y CONTROL DE VERSIONES 6

5.1. Flujo de trabajo (Gitflow) 6

5.2. Integración y revisión de código 6

5.3. Estructura del repositorio 6

6. ELEMENTOS ORIGINALES Y OPCIONALES 6

6.1. Personalización temática 6

6.2. Funcionalidades extra 6

7. CONCLUSIONES 6

1. INTRODUCCIÓN

1.1. Contexto

Este proyecto se enmarca en la asignatura de Informática Industrial, como trabajo obligatorio a realizar en el tercer curso del Grado en Ingeniería Electrónica Industrial y Automática ofrecido por la Escuela Técnica Superior de Ingeniería y Diseño Industrial (ETSIDI) de la Universidad Politécnica de Madrid.

El proyecto consiste en el diseño y desarrollo en C++ de un videojuego estructurado bajo el paradigma de la Programación Orientada a Objetos (POO). Tomando como referencia las mecánicas fundamentales del clásico *Archon: The Light and the Dark* (1983) y adaptándolas al entorno moderno de Visual Studio 2026 junto a OpenGL (mediante *freeglut)* y la librería *ETSIDIlib* para el renderizado y gestión de recursos multimedia de realización propia.

El código ha sido concebido desde su inicio respetando la normativa del proyecto, garantizando un fuerte desacoplamiento entre el motor gráfico y las estructuras lógicas, aplicando patrones de diseño y gestionando los recursos dinámicos mediante el patrón RAII (Resource Acquisition Is Initialization) para asegurar la robustez y evitar fugas de memoria.

1.2. Origen de "Rancho"

Rancho nace como una reinterpretación original del videojuego clásico *Archon*. Tanto es así que, cambiando un par de letras en forma de anagrama, se puede pasar del nombre original al de este proyecto.

Archon: The Light and the Dark está ambientado en un universo de fantasía heroica donde criaturas mitológicas, hechiceros y monstruos se enfrentan en una eterna batalla entre las fuerzas de la luz y la oscuridad. En *Rancho*, esta épica confrontación se traslada a un entorno mucho más terrenal, rústico y cómico: una granja.

La solemne dualidad entre el bien y el mal se sustituye por la feroz rivalidad territorial entre los distintos animales de corral. El tablero de casillas representa las parcelas de la

granja que cambian de estado, y la arena de combate simula los caóticos encontronazos que ocurren cuando dos especies se disputan la misma parcela.

Esta temática no solo aporta un tono humorístico y desenfadado, sino que justifica a nivel de diseño de gameplay una rica variedad de estadísticas, habilidades y tipos de ataque. Las diferencias anatómicas y de comportamiento de cada animal encajan a la perfección con la necesidad técnica de diseñar una jerarquía polimórfica variada, permitiendo diferenciar, por ejemplo, los ataques de embestida de una cabra con los lanzamientos de huevo de una gallina.

1.3. Objetivos del proyecto

El objetivo principal de este trabajo es asentar y demostrar de forma práctica los conocimientos sobre Programación Orientada a Objetos adquiridos durante el curso, materializándolos en un programa estructurado, eficiente y libre de errores. Para alcanzar este propósito general, se establecen los siguientes objetivos alineados con la normativa del proyecto.

a) Desacople lógico-gráfico

Garantizar una separación estricta entre la lógica (estructuras de datos, validación de reglas, estados de colisión) y el motor de renderizado gráfico, de modo que los métodos de las clases del juego no dependan directamente de las librerías gráficas.

b) Aplicación de jerarquías polimórficas

Modelar las entidades del juego mediante una clase base abstracta Animal que delegue el comportamiento específico a sus clases derivadas mediante enlace dinámico y métodos virtuales, maximizando la escalabilidad y reutilización del código.

c) Implementación de mecánicas duales

Desarrollar y coordinar dos lógicas de juego independientes pero interconectadas: una fase estratégica por turnos apoyada en estructuras matriciales (Tablero), y una fase de acción en tiempo real (Arena) que incluya algoritmos de detección de colisiones (AABB) y cinemática en un espacio continuo 2D.

d) Gestión robusta de memoria

Aplicar rigurosamente el patrón RAII en la construcción y destrucción de pantallas, entidades y recursos multimedia, garantizando la ausencia de fugas de memoria durante el ciclo de vida del programa.

e) Metodología de desarrollo colaborativo

Emplear un sistema de control de versiones mediante el flujo de trabajo Gitflow. Esto incluye la gestión de temas mediante issues, el aislamiento de funcionalidades en ramas feature/, y la integración de código a través de Pull Requests, simulando un entorno de producción profesional.

f) Requisitos opcionales

Incluir algunos opcionales

2. MANUAL DE INSTRUCCIONES

2.1. Requisitos del sistema y dependencias

Instrucciones detalladas para configurar Visual Studio 2026, freeglut y ETSIDIlib (también cómo enlazar dependencias no caben en el .zip).

2.2. Guía de inicio

Pasos para compilar el proyecto.

2.3. Controles

División de inputs (WASD y Flechas), uso del menú/HUD.

2.4. Reglas del juego

3. ARQUITECTURA DEL SOFTWARE (POO)

3.1. Desacople entre lógica y motor gráfico

Explicación de cómo la lógica trabaja de forma independiente a freeglut/ETSIDIlib, que está solo en la clase Renderizador.

3.2. Patrones de diseño

Máquina de Estados y Patrón Comando en la clase coordinadora, Juego.

Gestión de memoria y recursos (RAII). Patrones GRASP.
Regla de los Tres/Cinco/Cero (falta decidir cual, creo que Cero es la mejor).
Principios SOLID.

3.3. Estructura de clases

Abstracción, Encapsulamiento, Herencia y Polimorfismo.

El coordinador (Juego) y la gestión de fases (Menú, Tablero, Batalla).

Jerarquía polimórfica: clase abstracta Animal y sus hijas (Gallina, Cerdo, etc.), usando métodos virtuales puros y enlace dinámico.

Estructuras de control: Tablero (matriz de punteros) y Arena (fase de acción).

4. LÓGICA DE JUEGO

4.1. Fase estratégica (Tablero)

Validación de movimientos, oscilación de casillas y gestión de turnos.

4.2. Fase de acción (Arena)

Algoritmos de colisión (AABB), confinamiento en los límites de la pantalla y gestión dinámica de proyectiles/ataques.

4.3. Las piezas (Animales)

Características de cada animal, tipo, vida… Foto del animal.

Para el granjero, habilidades equivalentes a los hechizos del Archon original.

5. METODOLOGÍA Y CONTROL DE VERSIONES

5.1. Flujo de trabajo (Gitflow)

Uso de ramas main, develop y feature/.

5.2. Integración y revisión de código

Explicación del sistema de Pull Requests

5.3. Estructura del repositorio

Distribución profesional de directorios (/include, /src, /assets, etc.).

6. ELEMENTOS ORIGINALES Y OPCIONALES

6.1. Personalización temática

Elementos únicos de "Rancho" (gráficos, música, sonidos).

6.2. Funcionalidades extra

Menú principal, créditos…

7. CONCLUSIONES

Retos superados.

Trabajo futuro, posibles mejoras…